ProEm: Toolchain and Verification Guide

From SystemVerilog to a Tiny Tapeout chip: simulation, lint, coverage, formal proofs, the ASIC flow and GitHub Actions

Paul Vincent Bezzina

17 October 2026

Abstract

This guide explains, from the beginning, every tool and file that takes ProEm from SystemVerilog to a chip, and how to run each check yourself. It assumes you know Verilog from Quartus and ModelSim, but not the open-source ASIC world. Each section explains one idea in general terms, then shows exactly where it appears in the repository. Part II covers simulation, Part III formal verification, Part IV the ASIC flow, timing and Tiny Tapeout. The results of running all of it on ISA v0.7.1 are in the verification report (`verification_report.pdf`); this guide is about how the tools work.

Contents

I Getting started **4**

1 The big picture **4**

2 The files in your repository **5**

3 YAML **5**
3.1 What YAML is . . . 5
3.2 `info.yaml` . . . 6

4 JSON and `src/config.json` **6**

5 One-time setup (Windows, with WSL) **7**

II Simulation and checks **7**

6 Simulation **7**
6.1 What a simulator does . . . 7
6.2 RTL and gate-level simulation . . . 7
6.3 Waveforms . . . 8
6.4 Which suite to use . . . 8

7 The C++ test suite (CXXRTL) **8**
7.1 What `scripts/run.sh` does . . . 8
7.2 Running it . . . 8
7.3 The helpers in `sim/bench.h` . . . 9
7.4 Writing a new test . . . 9

8 cocotb **10**
8.1 What cocotb is . . . 10
8.2 Reading a test . . . 11
8.3 Running it . . . 11
8.4 Writing a new test . . . 11
8.5 Gate-level simulation on your computer . . . 12

9 Lint **12**

10 Coverage **12**

11 Independent decode with sigrok **13**

III Formal verification **13**

12 What formal verification is **13**

13 The tools **14**

14 The four building blocks **14**

15 How the tool searches **15**

16 The harness pattern **15**
16.1 Why a separate harness . . . 15
16.2 A reference model . . . 15
16.3 Checking data with one tracked value . . . 16
16.4 The `.sby` file . . . 16

17 Running proofs and reading the results **17**
17.1 When a proof fails . . . 17
17.2 Checking that a proof has teeth . . . 18

18 The six ProEm proofs **18**
18.1 Host FIFO (`host_fifo_formal.sv`) . . . 18
18.2 Frame buffer (`frame_buffer_formal.sv`) . . . 18
18.3 UART transmitter (`uart_tx_formal.sv`) . . . 18
18.4 Branch unit (`branch_unit_formal.sv`) . . . 19
18.5 CRC unit (`crc_unit_formal.sv`) . . . 19
18.6 Line codec (`line_codec_formal.sv`) . . . 19

19 Pitfalls **19**

IV From RTL to chip **20**

20 The ASIC flow (LibreLane) **20**
20.1 1. Lint and synthesis (Verilator, Yosys) . . . 20
20.2 2. Floorplan and power grid . . . 20
20.3 3. Placement (OpenROAD) . . . 21
20.4 4. Clock tree synthesis (CTS) . . . 21
20.5 5. Resizing and timing repair . . . 21
20.6 6. Routing . . . 21
20.7 7. Antenna repair, fill and extraction . . . 21

20.8 8. Sign-off . . . 21
20.9 9. Output . . . 21

21 Timing, explained simply **22**
21.1 Setup and hold . . . 22
21.2 Corners . . . 22
21.3 Slew . . . 22

22 Processes and PDKs **22**

23 How Tiny Tapeout works **23**
23.1 Sharing one chip . . . 23
23.2 The pins . . . 23

24 GitHub Actions: what happens when you push **23**
24.1 Workflows . . . 23
24.2 ProEm's workflows . . . 24
24.3 Reading the results . . . 24

25 Git in five commands **25**

26 Every check, in one place **25**

Glossary **25**

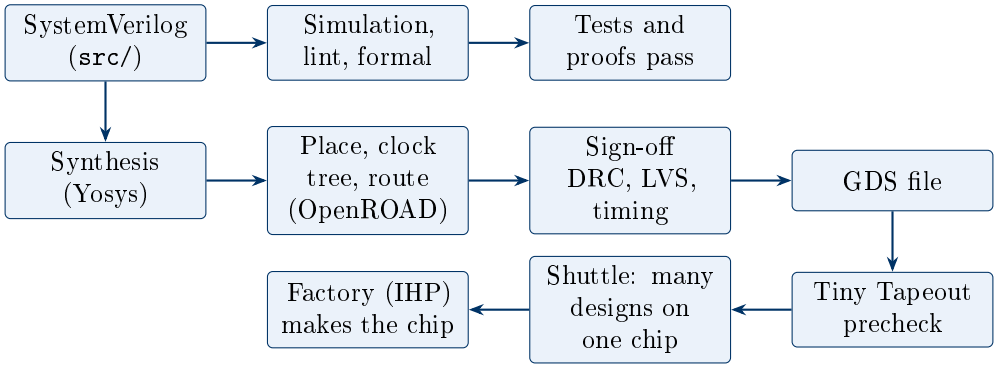
Part I

Getting started

1 The big picture

An FPGA is a chip that already exists; Quartus configures it. An ASIC is a chip that does not exist yet: the tools must decide where every transistor goes and draw the photographic masks a factory uses to make it. The output of that process is a **GDS file**, a drawing of every layer of the chip.

The journey for ProEm has two branches that meet at the end: one checks the design works (simulation and formal proofs), the other turns it into a layout (the ASIC flow).



Almost all of this runs automatically: when you push to GitHub, **GitHub Actions** (computers GitHub lends you) run the tests, the proofs and the whole ASIC flow, and show a green tick or a red cross. Your job is to write the SystemVerilog, keep the tests passing, and make sure the `gds` run is green. Jane Street's instructions say exactly this: start from the CMOS5L template, let the Tiny Tapeout Actions build the GDS and run the precheck, and submit a design with a passing build.

2 The files in your repository

File or folder	What it is for
`src/`	The SystemVerilog. `top/tt_um_vertrex_star_proem.sv` is the Tiny Tapeout wrapper; everything else sits under it. `src/files.f` lists every source file for the local scripts.
`src/config.json`	Settings for the ASIC flow (LibreLane): clock period, placement density, timing margins. Section 4.
`info.yaml`	The Tiny Tapeout project description: name, top module, tile size, source files, pin names. Section 3.
`test/`	The cocotb tests: `test.py`, `tb.v`, `Makefile`, `requirements.txt`. Section 8.
`sim/`	The C++ pin-level test suite: 16 test programs plus the shared `bench.h`, and `sim/verilator/` which lets the same tests run on Verilator. Section 7.
`formal/`	The formal proofs: one harness and one `.sby` file per module. Part III.
`lint/waivers.vlt`	The reviewed Verilator warnings. Section 9.
`scripts/`	`run.sh` (C++ suite), `lint.sh`, `coverage.sh`, `sigrok_check.py`, `formal.sh`, `synth_ihp.sh`, `sta.sh`.
`.github/workflows/`	What GitHub Actions run on every push: `gds.yaml`, `test.yaml`, `sim.yaml`, `formal.yaml`, `docs.yaml`. Section 24.
`docs/`	All documents (`docs/README.md` lists them), the datasheet page `info.md`, and the diagrams.
`gds/`	A copy of the finished layout (GDS, LEF), images, and a results summary.
`host/`	The Python host library (`proem_host.py`) that builds instruction bytes and decodes replies.

3 YAML

3.1 What YAML is

YAML is a plain-text format for writing down structured settings that both people and programs can read. It has only three building blocks:

- **A value**: text or a number, such as `50000000` or `"6x4"`.
- **A map** (also called a dictionary): names paired with values, written `name:  value`.
- **A list**: items written one per line, each starting with a dash `-`.

Nesting is shown by **indentation**, like Python. Everything indented under a line belongs to it.

```
project:              # a map called "project"...
  title: "ProEm"      # ...containing a value called "title"
  tiles: "6x4"
  source_files:       # ...and a list called "source_files"
    - "core/line_codec.sv"
    - "top/protocol_emulator.sv"
```

The rules that catch beginners:

- Indent with **spaces only**, never tabs. Two spaces per level is the convention.
- A colon must be followed by a space: `tiles:  "6x4"`, not `tiles:"6x4"`.
- `#` starts a comment; the rest of the line is ignored.

- Put text in quotes when it contains a colon, a `#`, or starts with a special character. Quoting all text, as `info.yaml` does, is the safe habit.

3.2 `info.yaml`

Tiny Tapeout reads this file to learn about the project. The important entries:

Entry	Meaning
`title`, `author`, `description`	Shown on the Tiny Tapeout website and in the printed datasheet.
`language: "SystemVerilog"`	Tells the tools how to read the sources.
`clock_hz: 50000000`	The clock wanted on the real chip (50 MHz). The demo board generates it.
`tiles: "6x4"`	How much of the shared chip ProEm uses: 6 tiles wide, 4 high. This picks the die outline, 1289.28 × 710.64 µm.
`top_module`	The module the flow builds. Must start with `tt_um_` and be unique: `tt_um_vertrex_star_proem`.
`source_files`	Every `.sv` file, relative to `src/`. A file missing here is invisible to the flow.
`pinout`	Names for each pin, used only for the datasheet and website. It does not change the hardware.
`yaml_version: 6`	The format version; never change it.

When you add a new SystemVerilog file, add it in **three** places: `info.yaml` (`source_files`), `test/Makefile` (`PROJECT_SOURCES`), and `src/files.f` (used by every local script).

The files in `.github/workflows/` are YAML too; Section 24 explains them.

4 JSON and `src/config.json`

JSON is YAML's stricter cousin: maps are written `{ "name": value }`, lists `[a, b]`, and every name is in double quotes. JSON has no comments, so the Tiny Tapeout template uses a trick: entries called `"//"` hold explanatory text, and the tools ignore them.

`src/config.json` configures LibreLane, the ASIC flow. The settings that matter:

Setting	Meaning
`CLOCK_PERIOD: 20`	The clock period in nanoseconds the flow must meet (20 ns = 50 MHz).
`PL_TARGET_DENSITY_PCT: 60`	How tightly cells are packed during placement. Higher packs cells closer (shorter wires) but makes routing harder.
`PL_RESIZER_HOLD_SLACK_MARGIN`, `GRT_RESIZER_HOLD_SLACK_MARGIN`	Extra safety the tool adds when fixing hold time (Section 21).
`MAX_TRANSITION_CONSTRAINT: 0.75`	Added for ProEm: no signal may take longer than 0.75 ns to switch (Section 21.3).
`PL_RESIZER_SETUP_SLACK_MARGIN: 3.0`, `GRT_RESIZER_SETUP_SLACK_MARGIN: 3.0`	Added for ProEm: the resizer aims for 3 ns of spare setup time rather than just enough, so the slow corner, which the flow does not enforce, passes too.
`RUN_POST_GRT_RESIZER_TIMING: 1`	Added for ProEm: repair setup again after global routing, when the wire lengths are known.
`FP_PDN_VPITCH`, `FP_PDN_VWIDTH`	Spacing and width of the power stripes; set by the CMOS5L template.

Everything below the template's "DO NOT CHANGE ANYTHING BELOW THIS POINT" line should be left alone: it matches the Tiny Tapeout chip.

5 One-time setup (Windows, with WSL)

Install WSL (Windows Subsystem for Linux) from the Microsoft Store ("Ubuntu"), open the Ubuntu terminal, and install everything the local checks need:

```
sudo apt update
sudo apt install -y iverilog g++ make git python3-pip python3-venv gtkwave verilator
    sigrok-cli
python3 -m venv ~/proem-env
source ~/proem-env/bin/activate        # run this line again in every new terminal
pip install cocotb==2.0.1 pytest yowasp-yosys
```

`yowasp-yosys` is Yosys packaged for Python, so you do not need to build it. Your Windows folders appear under `/mnt/c/`, for example `/mnt/c/Users/Paul/Documents/ProEm`. For the formal proofs you also need the OSS CAD Suite (Section 13).

Part II

Simulation and checks

6 Simulation

6.1 What a simulator does

A simulator runs your Verilog on a normal computer so you can check its behaviour before building anything. It keeps a list of events ("clock rises at 20 ns", "this signal changes") and processes them in time order. You met this in ModelSim. ProEm uses three open-source simulators.

Tool	Role in ProEm
Yosys CXXRTL	Turns the design into C++ so the 16 pin-level test programs in `sim/` run fast. The main regression.
Verilator	Lints the code (Section 9), and runs the same C++ tests a second time for coverage (Section 10).
Icarus Verilog	Runs the cocotb tests, on the RTL and on the gate-level netlist. Used by Tiny Tapeout's GitHub Actions.

6.2 RTL and gate-level simulation

- **RTL simulation** runs the SystemVerilog you wrote. Fast, easy to debug.
- **Gate-level (GL) simulation** runs the **netlist**: the list of real standard cells (AND gates, flip-flops) the flow produced. Each cell comes with a Verilog model from the PDK (`sg13cmos5l_stdcell.v`). It is slower, but it shows the flow did not change the design's behaviour, and it catches things RTL simulation hides, such as a register without a reset starting as `X`.

6.3 Waveforms

A waveform file records every signal over time, like ModelSim's wave window. Open it with **GTKWave** or **Surfer** (both free). FST is a compressed format; VCD is the older, larger equivalent. The cocotb tests write `test/tb.fst`; the C++ tests write a VCD of the pins when asked (Section 7).

6.4 Which suite to use

	C++ (CXXRTL)	cocotb (Python, Icarus)
Speed	Fast: thousands of programs in a minute	Slow: seconds to minutes per test
Waveforms	Pins only (`PROEM_VCD`)	Every signal, `test/tb.fst`
Gate-level netlist	No, RTL only	Yes (`GATES=yes`)
Run by GitHub	`sim` workflow	`test` and `gl_test` jobs
Best for	Many scenarios, the standards, random testing, quick checks after every change	Looking inside the design, debugging one scenario, checking the real netlist

A good habit: change the RTL, run the C++ suite (fast), and when something fails and you need to see why, reproduce it in cocotb and look at the waveform.

7 The C++ test suite (CXXRTL)

7.1 What `scripts/run.sh` does

1. Yosys reads every file listed in `src/files.f` and writes the whole chip as one C++ class into `build/emu_sim.h`. Each port becomes a member: `rx_serial` becomes `p_rx__serial` (`p_` in front, and each underscore doubled).
2. `g++` compiles each test in `sim/` together with that header, producing programs such as `build/t_ps2`.
3. It runs every test with the settings it needs, then (unless `SKIP_SYNTH=1`) synthesises the design to report its size.

The tests are pin level: `sim/bench.h` sends instruction bytes over the RX pin at a chosen baud rate, decodes what comes back on TX, and lets each test model a real device on the pins (an SPI device, an I^2C controller, a USB host). Together they cover the classic protocols, the 16 standards of the standards guide, the CRC and reply, branching, streaming, all four SPI modes, plus 1,800 random programs and 600 random branch programs.

7.2 Running it

```
cd /mnt/c/Users/Paul/Documents/ProEm
SKIP_SYNTH=1 bash scripts/run.sh      # all tests, a few minutes
bash scripts/run.sh                    # tests, then synthesis area
```

Each check prints a line ending in `PASS` or `FAIL` with what was received, and each program ends with `ALL PASS` or `FAILED`. The script stops at the first failure.

Run one test again. After one full run, the programs are in `build/`. The number after the name is the setting the test takes; `scripts/run.sh` shows the ones it uses:

```
./build/t_ps2 40          # PS/2 with a 40-clock half period
./build/t_spi 6           # SPI with SCK half period 6 clocks
./build/t_random 3 1 150  # seed 3, MSB first, 150 random programs
```

```
./build/t_standards dali  # one standard from the standards guide
```

Pin waveforms. Set `PROEM_VCD` to write a VCD of the eight protocol pins, the host RX and TX lines and both select outputs:

```
PROEM_VCD=build/dali.vcd ./build/t_standards dali
gtkwave build/dali.vcd
```

Rebuild after changing the RTL. Run `scripts/run.sh` again. It regenerates `emu_sim.h` and recompiles everything, which is the simplest way to be sure nothing is stale.

7.3 The helpers in `sim/bench.h`

Helper	What it does
`power_on()`	Resets the chip and sends 0x55 for auto-baud.
`CH = 0` or `1`	Which channel the next instructions address.
`I_line`, `I_clock`, `I_start`, `I_hdr`, `I_main`, `I_end`, `I_data`, `I_branch`, `I_run`, and the rest	Build one instruction's bytes, following the ISA.
`cat({...})`	Joins several instructions into one list.
`program(bytes)`	Sends the bytes to the chip over RX and waits until they are delivered.
`idle(n)`	Runs `n` clock cycles.
`env` and `pin_in`	`env` is a function called every clock cycle; set `pin_in` inside it to drive the protocol pins, as a connected device would.
`t.p_data__out__a.get<bool>()`	Reads a chip output (here channel A's data pin value).
`wait_return()`	Runs until the chip has finished sending results.
`got[c]`, `status[c]`	What channel `c` sent back: data as (bits, value) pairs, and one status byte per frame.
`report(name, ok, detail)`	Prints one `PASS` or `FAIL` line.

7.4 Writing a new test

This test sends one UART byte (0xA5) into pin 0 and checks that channel A reports it. Save it as `sim/t_my_uart.cpp`:

```
#include "bench.h"
int line_level = 1;
void my_env() { pin_in = 0xFE | line_level; }   // pin 0 is our UART line

void uart_send(unsigned b, int r) {
  int bits[10]; bits[0] = 0;                     // start bit
  for (int i = 0; i < 8; i++) bits[1 + i] = (b >> i) & 1;
  bits[9] = 1;                                   // stop bit
  for (int i = 0; i < 10; i++) { line_level = bits[i]; idle(r); }
}

int main() {
  env = my_env; power_on();
  int r = 16; CH = 0;
  program(cat({I_line(0, 0), I_clock(ASYNC, r), I_start(0, 1, 0, 0, 0),
               I_main(9, 0), I_run(0)}));
  uart_send(0xA5, r);
  wait_return();
```

```
  V exp = {{8, 0xA5}, {1, 1}};
  bool ok = report("UART receive of one byte",
                   got[0] == exp && status[0] == std::vector<unsigned>({1}),
                   show(got[0]) + " |" + showst(status[0]));
  printf(ok ? "ALL PASS\n" : "FAILED\n");
  return !ok;
}
```

To compile and run it by itself (after `scripts/run.sh` has created `build/emu_sim.h` once):

```
RT=$(python3 -c "import yowasp_yosys,os;print(os.path.join(os.path.dirname(yowasp_yosys.
    __file__),'share/include/backends/cxxrtl/runtime'))")
g++ -std=c++14 -O2 -I"$RT" -Ibuild sim/t_my_uart.cpp -o build/t_my_uart
./build/t_my_uart
```

It prints `UART receive of one byte ...  8:A5 1:01 | 01 PASS` and `ALL PASS`. To make it part of the suite, add `t_my_uart` to the `TESTS` list in `scripts/run.sh` (and in `scripts/coverage.sh`) and add a line `./build/t_my_uart` with the others.

Random tests and seeds. `t_random` and `t_random_branch` generate random programs from a **seed** (a starting number for the random generator). The same seed always gives the same programs, so when one fails, rerun it with that seed to reproduce the failure exactly.

Common problems.

- `yosys: command not found`: activate the environment: `source ~/proem-env/bin/activate`.
- `emu_sim.h: No such file`: run `scripts/run.sh` once before compiling a test by hand.
- A test hangs: it is waiting for a reply that never comes. Check the program (wrong channel in `CH`, missing `I_run`) and that `env` drives the pins the test expects.
- A new source file is not seen: add it to `src/files.f` (and to `info.yaml` and `test/Makefile`).

8 cocotb

8.1 What cocotb is

cocotb ("coroutine co-simulation testbench") lets you write the testbench in **Python** instead of Verilog. The simulator (Icarus) runs the hardware; cocotb runs your Python alongside it and can read and drive any signal.

File	Role
`tb.v`	A small Verilog top level that instantiates the chip, adds wiring the tests need (for example, looping channel B's output back to channel A, and an open-drain I^2C bus), and starts the waveform dump.
`test.py`	The six tests: auto-baud and errors, UART loopback, SPI loopback, I^2C target loopback, UART at 0x0000, CRC append and check. Each function marked `@cocotb.test()` is one test.
`Makefile`	Lists the source files and includes cocotb's own make rules, which call Icarus. `GATES=yes` switches to the gate-level netlist.
`requirements.txt`	The Python packages to install (`cocotb==2.0.1`, `pytest`).

8.2 Reading a test

```
@cocotb.test()
async def test_uart_loopback(dut):
    link = await power_on(dut)          # start clock, reset, send 0x55
    dut.loop_uart.value = 1             # wire B's output into A's input
    ...
    await link.send(program + link.run(0))    # program the chip
    await ClockCycles(dut.clk, 3 * 10 * r + 200)  # let it run
    assert link.got[0] == expected, f"channel A read {link.got[0]}"
```

- `dut` ("device under test") is a handle to `tb.v`; `dut.ui_in.value = 1` drives a pin, `dut.uo_out.value` reads one.
- `async` and `await` are how cocotb waits for simulated time. `await ClockCycles(dut.clk, 100)` pauses the Python until 100 clock edges have passed.
- `assert` checks a result; if it is false the test fails and prints the message.
- The `Link` class in `test.py` turns calls such as `link.main(9, 0)` into instruction bytes, sends them bit by bit on `ui_in[0]`, and decodes the chip's replies from `uo_out[0]` into `link.got` and `link.status`.

8.3 Running it

```
cd /mnt/c/Users/Paul/Documents/ProEm/test
make -B                                     # all six tests on the RTL
make -B COCOTB_TEST_FILTER=test_spi_loopback  # one test
gtkwave tb.fst                              # the waveform
```

`-B` forces a rebuild, so the latest RTL is always used. The summary line reads `TESTS=6 PASS=6 FAIL=0`, and `results.xml` is what the GitHub `test` workflow checks. In GTKWave, expand `tb` and then `user_project` to reach the design's hierarchy (`core`, `channel_a`, and so on). `uo_out[0]` is the chip's TX line, `ui_in[0]` is RX, and `uio_out` and `uio_oe` are the protocol pins.

8.4 Writing a new test

The same UART check as the C++ example, added to the end of `test/test.py`:

```
@cocotb.test()
async def test_my_uart_byte(dut):
    link = await power_on(dut)          # clock, reset, auto-baud
    r = 16                              # 16 clocks per UART bit
    link.channel = 0
    await link.send(link.line(0, 0)     # plain line coding on pin 0
                    + link.clock(ASYNC, r)
                    + link.start(0x00, 1, 0, 0, 0)   # wait for the start bit
                    + link.main(9, 0)   # read 9 bits: 8 data + stop
                    + link.run(0))
    byte = 0xA5
    for level in [0] + [(byte >> i) & 1 for i in range(8)] + [1]:
        dut.ext_in.value = 0xFE | level # pin 0 carries our UART line
        await ClockCycles(dut.clk, r)
    await link.quiet()                  # wait until the chip stops replying
    dut._log.info(f"channel A read {link.got[0]}, status {link.status[0]}")
    assert link.got[0] == [(8, 0xA5), (1, 1)], f"got {link.got[0]}"
    assert link.status[0] == [1], f"status {link.status[0]}"
```

Run it with `make -B COCOTB_TEST_FILTER=test_my_uart_byte`. The log shows `channel A read [(8, 165), (1, 1)], status [1]`: 8 bits with value 165 (0xA5), then the stop bit, then status 0x01 (frame complete). `dut.ext_in` is a register in `tb.v` that drives the protocol pins from outside, like a device connected to the chip.

Reading internal signals. Any signal can be read from Python, for example

```
dut.user_project.core.channel_a.chain.base_mode.value
```

Reading a signal that is `X` and converting it with `int(...)` raises "Can't convert LogicArray to int". That usually means a register without a reset, or a signal read before reset finished.

8.5 Gate-level simulation on your computer

This is optional, because GitHub's `gl_test` job does it on every push.

1. From a green `gds` run, download the `tt_submission` artifact. Inside it, `tt_um_vertrex_star_proem.v` is the routed netlist; copy it into `test/` as `gate_level_netlist.v`.

2. Install the CMOS5L PDK at the revision the CI uses:

   ```
   mkdir ~/pdk && cd ~/pdk && git init -q
   git fetch --depth 1 https://github.com/IHP-GmbH/IHP-Open-PDK.git 2
       bbec755dc67ca3db0261c3d6163e15735d66710
   git checkout -q FETCH_HEAD
   ```

3. Install Tiny Tapeout's Icarus 13. Ubuntu's Icarus 12 produces `X` values with the CMOS5L cell models, so the test fails even though the design is fine:

   ```
   wget https://github.com/TinyTapeout/iverilog/releases/download/v13.0/iverilog_13
       .0-1_amd64.deb
   sudo apt install -y ./iverilog_13.0-1_amd64.deb
   ```

4. Run it: `cd test && make -B GATES=yes PDK_ROOT=$HOME/pdk`. This takes several minutes, because every one of the 25,600 cells is simulated.

9 Lint

A **linter** reads the code without simulating it and reports likely mistakes: width mismatches, signals that are never used, case statements that miss a value, combinational loops. The ASIC flow runs Verilator's linter with the default warnings; `scripts/lint.sh` runs it with every warning on (`-Wall`):

```
bash scripts/lint.sh
lint clean: verilator -Wall, 13 reviewed waivers
```

Each warning was read and either fixed or recorded in `lint/waivers.vlt` with the file and line it applies to, for example an unused status output the top level does not route. A new warning makes the script fail, so it cannot slip in unnoticed. The `sim` workflow runs it on every push.

10 Coverage

Code coverage measures which lines and branches of the RTL the tests actually exercise. A line that no test reaches is a line no test checks. `scripts/coverage.sh` builds a Verilator model of the design with coverage counters, runs the whole C++ suite on it through `sim/verilator/emu_sim.h`

(an adapter that makes the Verilator model look like the CXXRTL one, so the tests are unchanged), merges the counts and annotates the source:

```
bash scripts/coverage.sh
Total coverage (1344/1361) 98.00%
See lines with '%00' in build/cov/annotated
```

Search the files in `build/cov/annotated/` for `%00` to find the points never reached. ProEm's 17 are listed in the verification report: most are `default` branches that cannot be reached, the rest are corner cases no standard needs. Running the tests on a second simulator is a check in itself: a test that passes on CXXRTL but fails on Verilator points at a race or an uninitialised value.

11 Independent decode with sigrok

The C++ device models and the RTL were written by the same person, so a misreading of a standard could pass both. **sigrok** is the open-source logic analyser software, with protocol decoders written independently for each standard. `scripts/sigrok_check.py` records the pins of each standards test as a VCD, decodes it with `sigrok-cli`, and compares the bytes with values computed separately in Python (CRC-16 Modbus, NMEA checksum, LIN checksum, SMBus PEC):

```
SKIP_SYNTH=1 bash scripts/run.sh       # builds build/t_standards
python3 scripts/sigrok_check.py        # all 30 checks
python3 scripts/sigrok_check.py dali   # one standard
  DALI forward frame (address 03, command 90)    PASS   Raw data: 03 | Raw data: 90
```

A failing check prints what the decoder saw. The same decoders run in PulseView against a real logic analyser, so the checks carry over to hardware unchanged.

Part III

Formal verification

12 What formal verification is

A simulation test drives chosen inputs and checks the outputs. However many tests you write, you only learn about the input sequences you tried. 1,800 random programs are a lot, but a FIFO bug that needs "write, flush, read and write in the same cycle, while full" may never be generated.

A formal tool works the other way round. You state rules that must always be true (**assertions**), and the tool searches *every* possible input sequence, mathematically, for one that breaks a rule. Either it proves that none exists, or it hands you the shortest input sequence that breaks it, as a waveform (a **counterexample**).

	Simulation	Formal
Inputs	The ones your test chooses	All possible inputs, every cycle
Answer	"No failure seen"	"Proven" or "here is a failing trace"
Best for	Whole-chip behaviour, long protocols	Control logic: FIFOs, pointers, state machines, arbitration
Weak at	Rare corner cases	Very large designs, long counters, arithmetic

The two complement each other. ProEm uses simulation for the protocols and the whole chip, and

formal proofs for the blocks whose corner cases are hardest to reach by testing. The line codec proof found a real bug (a lost `tx_done` after a cancel) that 1,800 random programs had missed.

13 The tools

Tool	Role
Yosys	Reads the SystemVerilog and the formal harness and turns them into a mathematical model. The same tool does synthesis.
SymbiYosys (SBY)	The driver. It reads a small `.sby` file, runs Yosys, calls the solver, and reports PASS or FAIL.
yices, boolector, bitwuzla, z3	**SMT solvers**: programs that answer "is there any assignment of values that makes this formula true?". SBY uses them through `yosys-smtbmc`. yices is usually the fastest for hardware.
ABC (`pdr`)	A model checker that proves properties with no depth limit by finding an invariant automatically (Section 15).
GTKWave or Surfer	Viewers for counterexample and cover waveforms (`.vcd`).

The simplest way to get all of them, at matching versions, is the **OSS CAD Suite**, a single download from YosysHQ. In the Ubuntu (WSL) terminal, open the releases page,
https://github.com/YosysHQ/oss-cad-suite-build/releases,
copy the link to the newest `oss-cad-suite-linux-x64-YYYYMMDD.tgz`, then:

```
cd ~
wget <the link you copied>
tar -xzf oss-cad-suite-linux-x64-*.tgz
source ~/oss-cad-suite/environment    # run this in every new terminal
sby --help                             # check it works
```

To make the `source` line automatic, add it to the end of `~/.bashrc`. The `formal` GitHub workflow installs the same suite with the `YosysHQ/setup-oss-cad-suite` action.

14 The four building blocks

A formal harness is ordinary SystemVerilog plus four kinds of statement.

`assert`: what must always be true.

```
assert (full == (count == DEPTH));
```

The tool tries to make this false. If it cannot, the property is proven.

`assume`: what the environment promises.

```
initial assume (!rst_n);        // the design starts in reset
always_comb assume (ratio != 0); // the divider is never programmed to 0
```

Assumptions restrict the inputs the tool may choose. They are powerful and dangerous: an assumption that is too strong can rule out the very case that would fail, and the proof then passes without meaning anything. Keep them few, and only for things the rest of the chip really guarantees.

`cover`: proof that something can happen.

```
cover (full);   // show me a trace where the FIFO fills up
```

A cover asks the tool to *find* a trace where the condition is true, and writes it as a waveform. Covers catch **vacuous** proofs: if the FIFO can never become full under your assumptions, every assertion about "when full" passes trivially, and the cover exposes that.

Free values: inputs, `anyconst`, `anyseq`. Every input port of the harness is free: the tool may give it any value in any cycle. For a value that is chosen once and then held, mark a signal `(* anyconst *)`:

```
(* anyconst *) logic [3:0] ratio;   // any divider from 0 to 15, fixed for the run
```

That single line makes the UART proof cover every clocks-per-bit setting at once.

15 How the tool searches

Bounded model checking (`mode bmc`)
Checks every input sequence up to a fixed length, say 30 cycles from reset. It finds real bugs quickly, but it says nothing about cycle 31, and the work grows quickly with depth.

k-induction (`mode prove` with `smtbmc`)
Proves "if the assertions held for k cycles in a row, they hold in the next one too" from *any* state, not only reachable ones. It often fails on harmless designs, because the tool starts from impossible states (a harness counter that disagrees with the FIFO's pointers). Fixing this needs extra assertions about internal signals, which a separate harness cannot reach.

PDR, also called IC3 (`mode prove` with `abc pdr`)
Builds the missing invariant automatically, by learning which states are unreachable. For control logic like ProEm's buffers and state machines it proves properties with **no depth limit**. This is what ProEm's proofs use.

Covers always run as a bounded search, since they only need one example.

16 The harness pattern

Each proof has two files in `formal/`: a harness (`<module>_formal.sv`) and a job description (`<module>.sby`). The RTL in `src/` is not modified.

16.1 Why a separate harness

- The RTL stays exactly as it is synthesised; nothing is added under `ifdef`s.
- The harness only uses the module's ports, so the properties describe the behaviour the rest of the chip relies on, not how the module is built. If you rewrite the module, the same proof still applies.
- The harness can shrink parameters (a 4-entry buffer instead of 16) to keep the proof fast. The logic is the same for any size.

16.2 A reference model

The harness keeps a small model of what the module *should* be doing, built only from its inputs and outputs, and asserts that the module agrees with it. For the host FIFO the model is a single counter:

```
assign wrFire = wr_en && !full && !flush;
assign rdFire = rd_en && !empty && !flush;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= '0;
    end else if (flush) begin
```

```
        count <= '0;
    end else begin
        count <= count + wrFire - rdFire;
    end
end

always_comb begin
    if (started && rst_n) begin
        assert (count <= DEPTH);
        assert (empty == (count == 0));
        assert (full == (count == DEPTH));
    end
end
```

`started` is a flag that is 0 in the very first cycle, so no assertion is checked before reset has been applied.

16.3 Checking data with one tracked value

Counting proves the flags, but not that data comes out unchanged and in order. Checking every entry would need a model as large as the FIFO. The standard trick is to follow *one* value, chosen by the tool:

- A free input `pick` marks the write the tool wants to follow. The harness records its value and how many entries are ahead of it.
- Every read moves it one place closer to the front.
- When it reaches the front, the FIFO's output must equal the recorded value.

```
if (!tracking && wrFire && pick) begin
    tracking <= 1'b1;
    trackedValue <= din;
    trackedPos <= count - rdFire;
end
...
if (tracking && trackedPos == 0) begin
    assert (dout == trackedValue);
end
```

Because the tool may pick *any* write, with *any* value, at *any* time, proving it for the one tracked value proves it for all of them.

16.4 The `.sby` file

```
[tasks]
prove
cover

[options]
prove: mode prove
cover: mode cover
cover: depth 30

[engines]
prove: abc pdr
cover: smtbmc yices
```

```
[script]
read -formal host_fifo.sv host_fifo_formal.sv
prep -top host_fifo_formal
memory_map

[files]
../src/primitives/host_fifo.sv
host_fifo_formal.sv
```

`[tasks]`
: Two jobs from one file: `prove` (the assertions) and `cover` (the covers). A line starting `prove:` only applies to that task.

`[options]`
: The mode of each task, and the search depth for the cover.

`[engines]`
: PDR for the proof, and yices through `smtbmc` for the covers.

`[script]`
: The Yosys commands. `read -formal` enables `assert`, `assume` and `cover`. `prep -top` picks the harness as the top module. `memory_map` turns the FIFO's storage array into ordinary registers, which the solvers handle much faster.

`[files]`
: The files SBY copies into its work folder.

SBY also converts asynchronous resets for you, so `always_ff @(posedge clk or negedge rst_n)` needs no special handling.

17 Running proofs and reading the results

```
bash scripts/formal.sh              # all proofs
cd formal && sby -f host_fifo.sby   # one proof, both tasks
sby -f host_fifo.sby prove          # one task only
```

`scripts/formal.sh` prints one line per module:

```
PASS  branch_unit (prove and cover)
PASS  crc_unit (prove and cover)
PASS  frame_buffer (prove and cover)
PASS  host_fifo (prove and cover)
PASS  line_codec (prove and cover)
PASS  uart_tx (prove and cover)
```

Each task writes a folder such as `formal/host_fifo_prove/`. The last lines of its `logfile.txt` say `DONE (PASS, rc=0)` or `DONE (FAIL, rc=2)`.

17.1 When a proof fails

The log names the failing assertion by file and line, and the folder holds a counterexample:

```
Assert failed in frame_buffer_formal: frame_buffer_formal.sv:113 ...
counterexample trace: frame_buffer_prove/engine_0/trace.vcd
```

Open `trace.vcd` in GTKWave. It is the shortest input sequence, from reset, that breaks the assertion. Read it like a failing simulation: find the cycle where the assertion fails, then walk back to see which input caused it. Then decide whether the RTL is wrong, or the property or an assumption is.

17.2 Checking that a proof has teeth

A proof that cannot fail is worthless, so plant a bug on purpose, on a copy of the RTL, and confirm the proof catches it:

Planted bug	Result
Frame buffer: rollback restores the write pointer to `cm_ptr + 1` instead of `cm_ptr`	FAIL at once, with a trace
Branch unit: rule 1 given priority over rule 0	FAIL at once, with a trace
CRC unit: the chunk loop takes one bit more than `feed_len`	FAIL at once, with a trace

Covers do the same job in the other direction: each cover must be *reached*, or the property it guards may be vacuous.

18 The six ProEm proofs

All are unbounded (PDR). Most take a few seconds; the line codec takes about 20 s and the CRC unit about four minutes.

18.1 Host FIFO (`host_fifo_formal.sv`)

Occupancy never exceeds the depth; `full` and `empty` match the count exactly; `flush` empties it; any value written comes out unchanged and in order. Covers: the FIFO fills; a tracked value travels the whole depth.

18.2 Frame buffer (`frame_buffer_formal.sv`)

The model keeps two counts: entries written and not yet read (`total`), and entries committed and not yet read (`committed`). It proves:

- `committed` $\leq$ `total` $\leq$ depth, and `full`, `almost_full` and `empty` are exact (`empty` is based on committed entries only, so the reader never sees a half-built frame);
- rollback discards exactly the uncommitted entries (a tracked value that was not yet committed is dropped; a committed one survives);
- `commit_all` includes a write in the same cycle;
- a tracked value appears on `dout` the cycle after it is read (the output is registered). The memory write is delayed by one clock with a bypass on the read side; the proof covers that too, since it only looks at the ports.

18.3 UART transmitter (`uart_tx_formal.sv`)

With `ratio` free (`anyconst`, 1 to 15), the model counts bits and cycles, and the proof shows every frame is a start bit, eight data bits LSB first and a stop bit, each held for exactly `ratio` clocks; the line is high whenever idle; and `tx_ready` is high exactly when idle. Cover: a full frame of 0xA5 at 3 clocks per bit.

18.4 Branch unit (branch_unit_formal.sv)

The harness models the two BRANCH rules from the ISA, loaded through the same configuration writes: compare the chosen block's byte under the mask with the value; take the hit or miss action; for a read-back rule, fire on a mismatch; rule 0 wins over rule 1; each rule acts at most once per frame, until the next frame starts or the rule is rewritten. It proves that `set_mode` and `new_mode` give the chosen action in the same cycle and only inside a frame; that `hit` and `branch_abort` follow one cycle later and are suppressed by a frame reset; that `stall` is high for exactly the decision cycle and the abort cycle; and that `line_watch` is set exactly when a read-back rule is enabled. A rule with `SRC` = 1 and `BLOCK` = 1 fires on the CRC result event and takes its hit action exactly when the CRC was good. The branch unit reference (`branch_unit.pdf`) explains the module itself.

18.5 CRC unit (crc_unit_formal.sv)

The chip keeps the CRC register top aligned in 16 bits and updates it with up to 8 bits per clock. The harness keeps a second, textbook model: a register of exactly the configured width, right aligned, updated one bit at a time with the polynomial masked to that width. Width, polynomial, initial value, residue, inversion and every input are free, so the proof covers every CRC of 1 to 16 bits. It proves:

- the chip's register is always the model shifted to the top (`crc_out`, inverted when asked);
- `good` is exactly "model equals residue";
- feeding the register its own value, top bit first, always leaves zero. This is why a CRC appended without inversion checks with residue 0 at the receiver;
- the configuration outputs (`enabled`, `check`, `append`, `width`, `cover_from`) match the last configuration write.

The assertions are skipped for the cycle after a configuration write, while the register reloads. Covers: a 16-bit CRC good after three chunks; a 5-bit CRC with the USB residue `0x0C`; an inverted output of all ones.

18.6 Line codec (line_codec_formal.sv)

The harness counts entries the codec has accepted and not yet answered, and leaves every line, clock and output configuration free and allowed to change at any time. It proves:

- `tx_done` never pulses with nothing outstanding, and at most four entries are ever outstanding;
- once the select output has been inactive for three clocks, every accepted entry has had exactly one `tx_done`, including entries dropped by any pattern of cancels;
- with `DRIVE` = `00` no output enable is ever set; with open drain the data and clock outputs only pull low; the clock output is only enabled while the codec generates the clock.

Covers: a cancel with two entries outstanding, a Manchester entry and an I^2C entry. The script flattens the design and adds `setundef -anyseq` (see Pitfalls). The line codec reference (`line_codec.pdf`) explains the module.

19 Pitfalls

Too-strong assumptions
An assumption that removes real inputs makes a proof meaningless. Every assumption should describe something the surrounding logic truly guarantees. Covers help catch this.

Checking during reset
Guard assertions with `started && rst_n`, or the tool will "fail" on the first cycle before reset has taken effect.

Memories
Arrays make the solvers slow or fail; `memory_map` in the script fixes this for small arrays.

Large parameters
Proof time grows quickly with widths and depths. Shrink parameters in the harness when the logic does not depend on them.

Long counters
A 16-bit baud counter has 65,536 states per bit. Constrain the value range, or shrink the counter width in the harness, as the UART proof does with a 4-bit ratio.

Undefined bits
Reading past the end of a vector gives `x` bits, and newer Yosys versions refuse them with "Design contains 'x' or 'z' bits". Pad vectors in the harness, and where the RTL itself does this behind a guard, add `setundef -anyseq` to the script, as `branch_unit.sby` and `line_codec.sby` do. `-anyseq` lets the bits take any value in every cycle, so the proof must hold whatever they are.

Solver differences
If one engine is slow, try another: `abc pdr` for unbounded proofs, `smtbmc yices` for bounded proofs and covers.

Not yet proven, and checked by simulation instead: the UART receiver (the auto-baud measurement makes the state space large), the bit counters of the frame chain, the bit-level timing of the line codec, and the control unit.

Part IV

From RTL to chip

20 The ASIC flow (LibreLane)

LibreLane (the successor of OpenLane) is a script that runs about 80 steps in order, calling several open-source programs. Each step's output is in `runs/wokwi/NN-step-name/`. The main stages, with ProEm's numbers from the v0.7.1 layout:

20.1 1. Lint and synthesis (Verilator, Yosys)

Verilator checks the code. Then **Yosys** performs **synthesis**: it turns the RTL into a netlist of **standard cells**, the pre-designed gates the factory offers (for example `sg13cmos5l_nand2_1`: a 2-input NAND, drive strength 1). This is what Quartus's "Analysis and Synthesis" does for LUTs; here the building blocks are real gates. ProEm becomes about 19,000 cells. `scripts/synth_ihp.sh` runs the same mapping locally, to compare the size of two versions of the RTL without running the whole flow.

20.2 2. Floorplan and power grid

The flow creates the empty die (6×4 tiles), places the pins where Tiny Tapeout expects them, draws rows for cells to sit in, and lays a **power distribution network** (PDN): metal stripes carrying the

supply (VPWR, 1.2 V) and ground (VGND) to every row. These are the regular vertical stripes in the layout pictures.

20.3 3. Placement (OpenROAD)

Every cell gets a position. Global placement spreads cells to keep connected ones close; detailed placement snaps them into legal row positions without overlap.

20.4 4. Clock tree synthesis (CTS)

ProEm's 2,763 flip-flops all need the clock at nearly the same moment. CTS builds a tree of buffers (362 cells) so the clock arrives everywhere with little **skew** (difference in arrival time).

20.5 5. Resizing and timing repair

The **resizer** swaps cells for stronger or weaker versions and inserts buffers to fix timing. On ProEm it adds about 6,100 repair buffers: delay cells (`dlygate4sd3`) to fix **hold** time, and buffers that split long, heavily loaded wires to fix **setup** time. This is why the cell count grows from about 19,000 to about 25,600.

20.6 6. Routing

Global routing plans which region each wire passes through; **detailed routing** draws every wire exactly, on the metal layers, obeying the factory's spacing rules. ProEm's wires total about 1.15 m. The router iterates until there are 0 violations.

20.7 7. Antenna repair, fill and extraction

- **Antennas**: during manufacturing, a long metal wire connected only to a transistor gate can collect charge and damage it. The flow adds small diodes where needed (51 on ProEm).
- **Fill**: empty space is filled with filler and decoupling cells so the chip is uniform to manufacture. Cells cover 43 % of the core; the rest is fill, which is why the layout looks full.
- **RC extraction**: the flow calculates the real resistance and capacitance of every wire (the `.spef` file) so the final timing check uses real wire delays.

20.8 8. Sign-off

Check	What it proves
STA (static timing analysis)	Every path meets setup and hold at every corner (Section 21).
DRC (design rule check), Magic and KLayout	The drawing obeys every manufacturing rule (minimum widths, spacings, overlaps). A single DRC error can make the chip fail.
LVS (layout versus schematic), Netgen	The transistors actually drawn connect exactly as the netlist says.
IR drop	The power grid delivers the supply without a large voltage loss (ProEm: 0.23 mV worst).

20.9 9. Output

The final GDS (`tt_um_vertrex_star_proem.gds`) is the drawing sent to the factory. The **LEF** is a simplified outline with only pin positions, which Tiny Tapeout uses to fit the block into the full chip. A copy of both, with pictures and the full results, is in `gds/`.

21 Timing, explained simply

21.1 Setup and hold

Between two flip-flops sits logic. When the clock ticks, the first flip-flop launches a new value; it travels through the logic and must reach the second flip-flop:

- **Setup**: early enough before the *next* clock edge. If the logic is too slow, the second flip-flop captures the old value. A setup failure can be fixed by slowing the clock.
- **Hold**: not so early that it overwrites the value being captured at the *same* edge. A hold failure cannot be fixed by slowing the clock, which is why the flow adds delay cells for it.

Slack is the spare time. Positive slack means the path passes; negative slack is a violation. The **maximum clock** is roughly 1/(period − setup slack): ProEm's slowest path takes $20 - 3.84 = 16.16$ ns at the slow corner, so about 62 MHz.

21.2 Corners

Real chips vary. Transistors are slower when hot, at low voltage, or when the factory makes them slightly "slow". Tools therefore check **corners** (process, voltage, temperature):

Corner	Supply	Temperature	ProEm setup slack	Hold slack
Slow	1.08 V	125 °C	3.84 ns	0.61 ns
Typical	1.20 V	25 °C	9.82 ns	0.29 ns
Fast	1.32 V	−40 °C	12.35 ns	0.10 ns

Setup is hardest at the slow corner; hold is hardest at the fast corner. The CMOS5L flow only *enforces* the typical corner, so a build can be green while failing the slow one. That happened to ProEm twice: in September (fixed with the slew limit below) and after the v0.7.1 additions, when the slow corner missed by 3.8 ns. The second time was fixed in the RTL, by registering the CRC unit's input and shortening the paths into the frame buffer and the CRC register, plus the larger setup margins in `config.json`. The verification report lists each step.

21.3 Slew

Slew (or transition time) is how long a signal takes to switch from 0 to 1. A small cell driving a long wire switches slowly, and a slow edge also slows every gate after it. In September ProEm's failing path had edges over 2 ns long; limiting slew to 0.75 ns forced the tool to use stronger cells and buffers.

22 Processes and PDKs

A **process** is a factory's recipe for making chips: the transistor size (here 130 nm, the length of the smallest transistor gate), the number of metal layers for wiring, and the supply voltages. A **PDK** (process design kit) is the package the factory gives designers: the rules (for DRC), the standard cells with their layouts and timing models (`.lib` files), and the Verilog models used in gate-level simulation. IHP, SkyWater and GlobalFoundries publish open PDKs, which is what makes Tiny Tapeout possible.

Process	Notes
IHP SG13CMOS5L	**The target for Jane Street.** 130 nm CMOS from IHP (Germany). Five routing layers: Metal1 to Metal4 plus one thick TopMetal1. Core supply 1.2 V, I/O 3.3 V. Cell names start `sg13cmos5l_`.
IHP SG13G2	The full IHP 130 nm process, with seven metal layers and fast bipolar transistors (for radio circuits). Same standard cells, more routing layers. `scripts/sta.sh` can use its timing libraries.
SkyWater SKY130	The first open PDK, used by earlier Tiny Tapeout shuttles.
GF180	GlobalFoundries 180 nm; another Tiny Tapeout option.

Fewer metal layers means fewer "floors" for wires, so routing is harder and wires are longer. That is one reason CMOS5L timing is a little slower than SG13G2 for the same design.

23 How Tiny Tapeout works

23.1 Sharing one chip

Making a chip costs far too much for one small design, so **Tiny Tapeout** puts hundreds of projects on one die. The die is divided into **tiles**; each project gets a rectangle of tiles (ProEm: 6×4). A batch of designs manufactured together is a **shuttle**. After manufacturing (several months), chips are mounted on a **demo board**. A small microcontroller on the board picks one project at a time through an on-chip multiplexer, supplies its clock and reset, and connects its pins to the board's headers.

23.2 The pins

Every project has the same interface, which is why the top module has exactly these ports:

Port	Meaning
`ui_in[7:0]`	8 inputs. ProEm uses `ui_in[0]` as RX from the host.
`uo_out[7:0]`	8 outputs. ProEm: TX, BAUD_LOCKED, ARMED, HOST_FULL, SELECT.
`uio_in`, `uio_out`, `uio_oe`	8 bidirectional pins. `uio_oe[i] = 1` makes pin i an output. ProEm's protocol pins are here, so every line can be driven and read back.
`clk`, `rst_n`	Clock, and reset (active low: 0 means reset).
`ena`	1 when this project is the one selected.

Projects start from Tiny Tapeout's template repository, which contains the workflows, `src/config.json`, a sample `info.yaml`, and the cocotb test layout. For Jane Street, the template is the `cmos5l` branch of Tiny Tapeout's IHP template; ProEm matches it.

24 GitHub Actions: what happens when you push

24.1 Workflows

A **workflow** is a YAML file in `.github/workflows/`. The line `on: push` means "run this every time someone pushes". A workflow has **jobs**; each job runs on a fresh Linux machine (`runs-on: ubuntu-24.04`) and has **steps**. A step either runs a command (`run:`) or uses a ready-made action (`uses:`). `needs: gds` means "wait for the `gds` job to finish first".

```
jobs:
  gds:
    runs-on: ubuntu-24.04
    steps:
```

```
      - name: Build GDS
        uses: TinyTapeout/tt-gds-action@ihp-cmos5l   # Tiny Tapeout's action, CMOS5L
    version
        with:
          pdk: ihp-sg13cmos5l
  precheck:
    needs: gds
    ...
```

24.2 ProEm's workflows

Workflow and job	What it does
`gds` / `gds`	Installs the CMOS5L PDK and LibreLane, runs the whole ASIC flow, and uploads the results. **This is the one Jane Street cares about.**
`gds` / `precheck`	Tiny Tapeout's own checks on the GDS: full KLayout DRC, pins in the right places, right outline, no forbidden layers.
`gds` / `gl_test`	Runs the cocotb tests on the routed gate-level netlist the `gds` job produced.
`gds` / `viewer`	Publishes a 3D viewer of the layout to GitHub Pages. On a private repository it fails (Pages needs a paid plan), so it is marked `continue-on-error`. It is not needed for submission.
`test`	The cocotb tests on the RTL.
`sim` / `simulate`	The C++ test suite, then the sigrok decode of the standards waveforms.
`sim` / `lint`	`scripts/lint.sh`: Verilator with every warning on.
`formal`	Every formal proof (`scripts/formal.sh`) with the OSS CAD Suite.
`docs`	Checks `docs/info.md` (the datasheet page) can be built.

24.3 Reading the results

On GitHub, open the **Actions** tab. Each push is a row; a green tick means every job passed. Click a run to see each job; click a job to see each step's log; the `gds` job's summary shows the cell counts, routing statistics and linter output. At the bottom of the run page, **Artifacts** are downloadable zips:

- `tt_submission`: the GDS, LEF, netlist and settings, the files that go onto the chip.
- `GDS_logs`: every step's reports, for debugging.
- `gds_render`: a picture of the layout.

If a job fails, open it and read the last red lines of the failing step. Common causes: a file missing from `info.yaml`, a Verilog error the linter reports, or a timing violation at the typical corner. A failure in the upload step ("Artifact storage quota has been hit") is not a design problem: the account's artifact storage is full. Delete old artifacts or wait for the quota to reset, then press **Re-run failed jobs** on the run page.

A private repository gets a monthly allowance of free Actions minutes and artifact storage. A full `gds` run takes tens of minutes, and the precheck DRC can take much longer, so avoid pushing many times in a row just to test small edits; run the local checks first.

25 Git in five commands

Command	Meaning
`git status`	What has changed since the last commit.
`git add file`	Choose a change to include in the next commit. `git add -A` includes everything.
`git commit -m "message"`	Save a snapshot with a description.
`git push`	Send the commits to GitHub; this triggers the Actions.
`git log -oneline`	List the commits.

A **commit** is a snapshot of the whole project. A **remote** (`origin`) is the copy on GitHub. A **tag** (`v0.7.1`) is a name for one commit. "Everything up to date" after `git push` means there were no new commits to send: you forgot `git add` and `git commit`. "Not a git repository" means the terminal is not in the folder that contains the hidden `.git` folder.

26 Every check, in one place

```
bash scripts/lint.sh                      # Verilator -Wall
SKIP_SYNTH=1 bash scripts/run.sh          # every pin-level test (CXXRTL)
cd test && make -B && cd ..               # cocotb, RTL
bash scripts/formal.sh                    # six proofs (OSS CAD Suite)
bash scripts/coverage.sh                  # Verilator coverage, report in build/cov
python3 scripts/sigrok_check.py           # needs sigrok-cli and build/t_standards
PDK_ROOT=~/pdk bash scripts/synth_ihp.sh typ_1p20V_25C   # cell count and area
```

Before submitting: push, wait for `gds`, `precheck` and `gl_test` to go green, and download the `tt_submission` artifact.

Glossary

ASIC
Application-specific integrated circuit: a chip made for one design, unlike an FPGA.

Assertion
A condition that must hold in every reachable state.

Assumption
A constraint on inputs; the formal tool only considers inputs that satisfy it.

BMC
Bounded model checking: all input sequences up to a fixed depth.

cocotb
Python testbench library that drives a Verilog simulator.

Corner
A combination of process, voltage and temperature at which timing is checked.

Counterexample
A trace, from reset, that breaks an assertion.

Cover
A condition the formal tool must show is reachable.

Coverage
The share of the RTL's lines and branches that the tests exercise.

CTS
Clock tree synthesis: building the buffer tree that distributes the clock.

CXXRTL
Yosys feature that turns Verilog into C++ for fast simulation.

DRC
Design rule check: the layout obeys the factory's geometric rules.

Fill cells
Cells with no logic, placed in empty space for manufacturability.

GDS
The file format of the chip layout sent to the factory.

Gate level
Simulation of the synthesised netlist of real cells rather than the RTL.

GitHub Actions
GitHub's machines that run the workflows on every push.

Harness
A wrapper module that instantiates the design and adds the model and properties.

Hold
Timing check that data does not change too soon after a clock edge.

Icarus Verilog
Open-source Verilog simulator (`iverilog`, `vvp`).

Induction
Proving that a property, if true for k cycles, stays true in the next.

Invariant
A property that holds in every reachable state.

LEF
Simplified outline of a block with its pin positions.

LibreLane
The open-source flow that runs synthesis to GDS.

Lint
Automatic check of code for likely mistakes without simulating.

LVS
Layout versus schematic: the drawn circuit matches the netlist.

Netlist
A design described as connected standard cells.

OpenROAD
The program that does floorplanning, placement, CTS, routing and timing.

PDK
Process design kit: rules, cells and models for a manufacturing process.

PDN
Power distribution network: the supply and ground wiring.

PDR / IC3
An algorithm that finds invariants automatically and proves properties without a depth limit.

Precheck
Tiny Tapeout's final checks before a design is accepted.

Reference model
A simple description of the intended behaviour, compared against the design.

RTL
Register-transfer level: the SystemVerilog you write.

SBY
SymbiYosys, the driver that runs Yosys and the solvers from a `.sby` file.

Setup
Timing check that data arrives early enough before the clock edge.

Shuttle
A batch of designs manufactured together.

sigrok
Open-source logic analyser software with independent protocol decoders.

Slack
Spare time on a timing path; negative means a violation.

Slew
How long a signal takes to switch.

SMT solver
A program that decides whether a logical formula over bit-vectors can be satisfied.

SPEF
File of extracted wire resistances and capacitances.

Standard cell
A pre-designed gate or flip-flop from the PDK.

STA
Static timing analysis: checking all paths' timing without simulation.

Synthesis
Turning RTL into a netlist of standard cells (Yosys).

Tile
The unit of area on a Tiny Tapeout chip (about $202 \times 155\,\mu\text{m}$ on IHP).

Utilisation
Share of the core area covered by cells (ProEm: 43 % after place and route).

Vacuous proof
A proof that passes only because the situation it talks about can never occur.

Verilator
Open-source Verilog linter and fast simulator.

Workflow
A YAML recipe that GitHub Actions runs.

YAML
Human-readable settings format based on indentation.

Yosys
Open-source synthesis tool.